

ALGORITHMIC TRADING SYSTEM ARCHITECTURE

*A robustness-first blueprint for daily / swing-horizon
equity trading with live capital*

Scope	Daily / swing equities, single-account, real-capital deployment
Design priority	Capital preservation and robustness over maximum predictive accuracy
Execution venue	Interactive Brokers (API-accessible, UK-eligible)
Document version	1.0

This document is a technical and risk-control blueprint, not financial advice. Markets carry genuine risk of capital loss; no architecture eliminates that risk — it only manages it.

Contents

1. Executive Summary
2. Design Principles
3. High-Level Architecture
4. Component 1 — Data Ingestion & Storage
5. Component 2 — Feature Engineering Layer
6. Component 3 — Predictive Model Ensemble
7. Component 4 — Signal Generation & Trade Selection
8. Component 5 — Risk Management Engine
9. Component 6 — Execution Layer
10. Component 7 — Backtesting & Validation Framework
11. Component 8 — Monitoring, Logging & Retraining
12. Technology Stack
13. Phased Build Roadmap
14. Risk Control Summary
15. Pre-Live Deployment Checklist
16. Appendix — Suggested Repository Structure

1. Executive Summary

This document specifies an end-to-end architecture for an algorithmic trading system intended to operate with real capital on a daily-to-swing horizon (roughly 1–20 trading days per position) in liquid equities. It is written to be built from scratch, in stages, by a single developer with a quantitative/ML background.

The architecture is deliberately conservative. At this horizon, in liquid equities, durable statistical edges are small and easy to mistake for noise. The design therefore optimises for **robustness and capital preservation** first, and predictive sophistication second — a system that survives contact with live markets and compounds a modest edge will outperform a more “clever” system that cannot survive its own overfitting.

The system has eight components: data ingestion, feature engineering, a model ensemble (gradient-boosted trees for direction, GARCH for volatility, a factor-exposure filter for false-alpha detection), a signal generator, a risk engine, an execution layer (Interactive Brokers), a backtesting/validation framework, and a monitoring/retraining layer. Each is independently replaceable, and every component includes an explicit failure mode and mitigation.

Deployment is staged: backtest → paper trade → small live tranche → gradual scale-up. No stage is skipped, regardless of how convincing the backtest looks.

2. Design Principles

Capital preservation first. Every component includes a fail-safe or fallback. A missing data feed, a model error, or a broker outage should degrade the system to “do nothing”, never to “do something unbounded”.

Simplicity over sophistication. Fewer moving parts means fewer ways to fail silently. A 3-model ensemble of well-understood techniques is preferred over a single deep, opaque model.

Walk-forward validation only. No single train/test split is trusted. Every backtest result is generated by rolling, strictly out-of-sample windows that respect the order of time.

One code path for backtest and live. Feature engineering and signal logic run from the same shared codebase in both backtest and live execution, eliminating the single most common source of backtest-live divergence.

Continuous, automatic monitoring. Drawdown limits, decay detection, and the kill switch are enforced programmatically, not left to manual discipline alone — manual discipline degrades exactly when it is needed most (during a losing streak).

3. High-Level Architecture

Data flows downward through ingestion, feature engineering, the model ensemble, signal generation, risk control, and execution. Two systems run alongside this primary flow: an offline backtesting/validation engine that continuously re-validates the model ensemble, and a monitoring layer that watches every stage in production and can trigger retraining or a full halt.

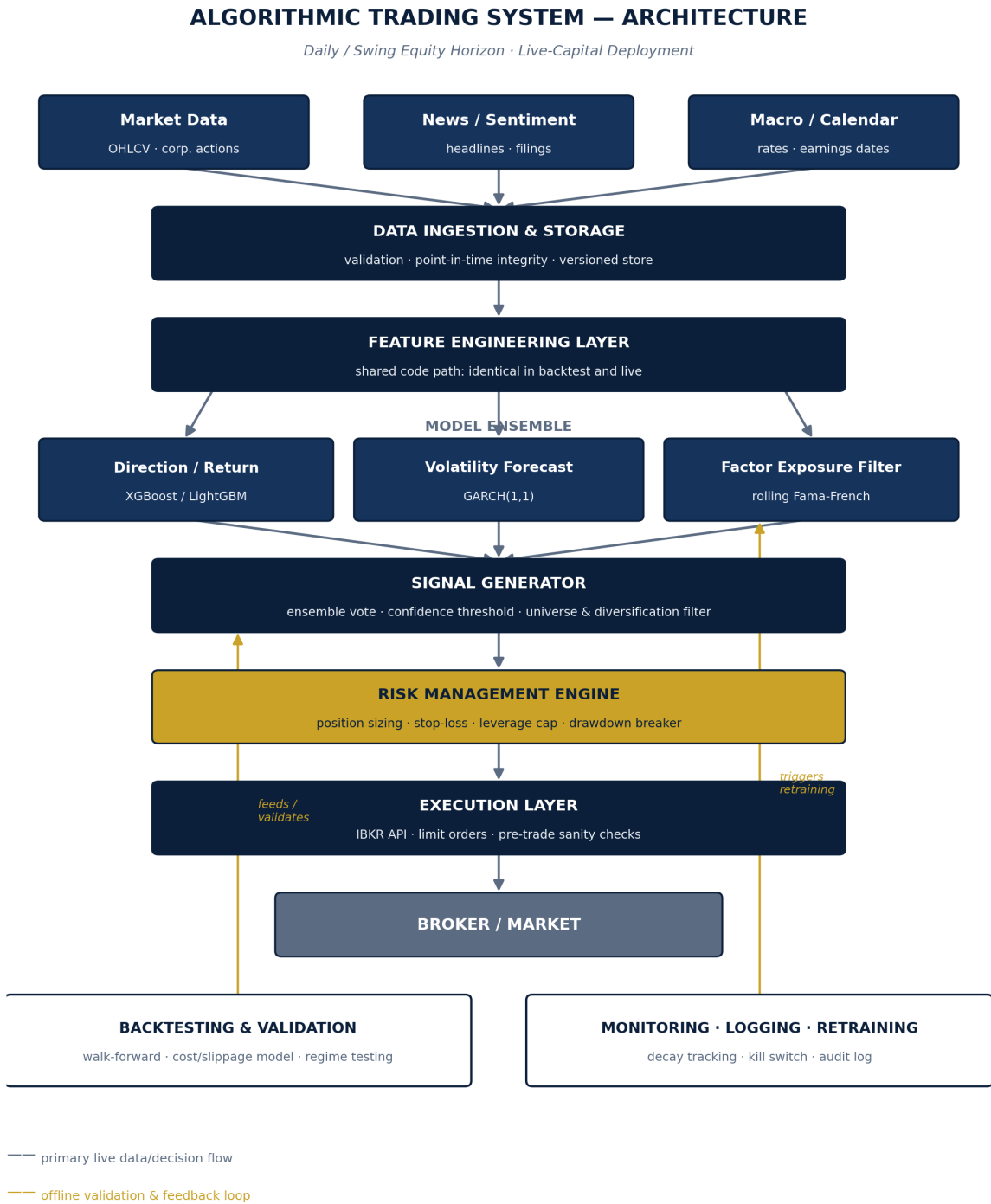


Figure 1. End-to-end system architecture and data/decision flow.

4. Component 1 — Data Ingestion & Storage

Purpose: Acquire, validate, and persist all raw data the system depends on, with strict point-in-time integrity.

Data sources

- **Market data:** OHLCV bars and corporate actions (splits, dividends) — via the IBKR market data API for live use, and a historical provider (e.g. Polygon.io, Alpha Vantage, or IBKR's own historical endpoint) for backtesting depth.
- **News / sentiment:** headline or filing feed (e.g. a news API or RSS aggregation), only if the sentiment feature is included in Component 2 — treat as optional, not core.
- **Macro / calendar:** earnings dates, rate-decision dates, index-rebalance dates — used to apply event blackout rules rather than as predictive features.

Storage

- A time-series database (PostgreSQL with TimescaleDB) for production scale, or partitioned Parquet files on disk for an initial single-developer build — either is fine to start; the schema matters more than the engine.
- Every dataset is versioned. Recomputing a feature set from a given data version must always reproduce the same result.

Critical implementation detail — point-in-time correctness

Two silent bugs destroy more retail trading systems than bad models do: **look-ahead bias** (using information not actually available at the decision timestamp — e.g. an “adjusted close” that incorporates a future split) and **survivorship bias** (backtesting only on stocks that still exist today, which inflates results). Every ingestion job must timestamp data with both its *event time* and its *availability time*, and downstream code must only ever query data available as of the decision timestamp.

Failure modes & mitigations

- *Stale or missing feed:* if the latest bar is older than a defined threshold, the system suspends new signal generation for the affected instrument rather than trading on stale data.
- *Corrupted/malformed data:* schema and sanity validation (price > 0, volume ≥ 0, no duplicate timestamps) runs before any write to storage; failed rows are quarantined and logged, not silently dropped.

5. Component 2 — Feature Engineering Layer

Purpose: Transform raw data into model-ready signals, using only information available at the decision timestamp.

Feature categories

Category	Examples
Price-based	N-day returns, momentum, RSI, moving-average crossovers, ATR-based realised volatility
Volume-based	Relative volume vs. rolling average, volume-weighted price deviation
Cross-sectional	Relative strength vs. sector and vs. index, sector rank
Factor exposure	Rolling Fama-French factor loadings (market, size, value, momentum)
Sentiment (optional)	Decayed/aggregated daily sentiment score from a transformer-based classifier (e.g. FinBERT)

Sentiment is included as one input feature into the direction model (Component 3), not as a standalone signal — in practice this kind of signal tends to be weak and short-lived on its own, and is more useful as a small

supporting input than as the basis of a strategy.

Implementation rule

The feature computation module is a single shared library imported by both the backtesting engine and the live system. It is never reimplemented twice. This is the single highest-leverage engineering decision in the whole architecture — most real-world backtest/live divergence traces back to two subtly different feature implementations rather than to the model itself.

6. Component 3 — Predictive Model Ensemble

Purpose: Generate a directional/return prediction, a volatility forecast, and a false-alpha filter — three narrow, well-understood models rather than one complex one.

Model	Technique	Role	Retraining
A — Direction / return	XGBoost / LightGBM classifier or regressor	Predicts probability of positive forward N-day return, or expected return magnitude, from technical + factor + (optional) sentiment features	Monthly, on a rolling 2–3 year training window
B — Volatility forecast	GARCH(1,1) / EGARCH	Forecasts forward volatility per instrument; feeds position sizing in the risk engine — it does not vote on direction	Re-fit weekly per instrument
C — Factor exposure filter	Rolling Fama-French regression	Rejects or down-weights candidate trades whose predicted return is statistically indistinguishable from pure market/factor beta	Re-fit monthly

Why not a single deep-learning model

Sequence models (LSTM/GRU, transformer-based architectures) can be included later as one additional voice in the ensemble, but are deliberately not the primary signal here. They require more clean data than a typical retail-scale dataset provides to outperform simpler models at this horizon, and their failure mode — confidently memorising noise — is exactly the failure mode this architecture is designed to avoid for live capital. They are a reasonable research extension once the simpler ensemble has a proven live track record.

Ensembling logic

- Model A produces a per-instrument confidence score for each candidate trade.
- Model C must independently confirm the candidate is not pure beta exposure — trades failing this check are rejected before they reach the signal generator.
- Model B's volatility forecast does not affect whether a trade is taken, only how large the resulting position is (Component 5).

7. Component 4 — Signal Generation & Trade Selection

Purpose: Convert model outputs into a small, diversified set of actionable trade candidates.

- **Universe filter:** restrict to liquid mid/large-cap equities initially (e.g. minimum average daily dollar volume threshold) — illiquid names introduce slippage that can erase the entire predicted edge.
- **Confidence threshold:** only act on the top decile (or other fixed percentile) of Model A's confidence scores, after the Component 3 factor filter — trading every prediction, including low-confidence ones, degrades signal-to-noise.

- **Diversification rule:** cap the number of concurrent open positions and the maximum exposure to any single sector, so that one regime shift cannot impact the whole book at once.
- **Event blackout:** candidates with an earnings release or major macro event inside the expected holding window are excluded or sized down, since these introduce a different (event-driven, gap-risk) return distribution than the model was trained on.

8. Component 5 — Risk Management Engine

Purpose: The component that does the most to protect capital. Model accuracy is rarely the reason live systems fail; gaps in this layer usually are.

Position sizing

Volatility-scaled fixed-fractional sizing: risk a fixed, small percentage of total capital per trade (commonly 0.5–1% in conservative live systems), with position size calculated as the risk amount divided by the instrument's current ATR (from Model B) multiplied by a stop-distance factor. This means a more volatile instrument automatically gets a smaller position for the same dollar risk — sizing is never set off conviction alone.

Trade-level controls

- Hard stop-loss on every position (e.g. $2 \times$ ATR from entry), enforced programmatically as a standing order at the broker, not just as an internal alert that depends on the system being online.
- Optional ATR-based trailing exit to let winners run while still bounding downside.

Portfolio-level controls

- Maximum gross and net exposure as a percentage of account equity.
- Maximum leverage cap, set explicitly before going live and never adjusted reactively after a winning or losing streak.
- Maximum sector concentration and maximum single-name weight.
- **Drawdown circuit breaker:** if rolling drawdown exceeds a defined threshold (e.g. -8% over a 20-trading-day window), the system automatically halts new trade entry pending manual review — existing positions are managed down via their stops, not force-liquidated in panic.

The circuit breaker and hard stop-losses are the two controls that most directly determine whether a string of losing trades stays a manageable drawdown or becomes a capital-threatening event. These should be implemented and tested before any other component is considered “done”.

9. Component 6 — Execution Layer

Purpose: Translate a sized, risk-approved trade decision into an actual market order, reliably and with minimal slippage.

Broker / venue

Interactive Brokers is the practical choice for a UK-based, API-driven system: it has a documented API (TWS API / IBKR Web API), supports UK accounts, and provides equities, ETFs and CFD access if leverage is used. Trading 212 does not currently expose a public trading API, so it cannot serve as the execution venue for this kind of automated system — it remains useful as a separate, manually-managed account if desired, but the live algorithmic capital should sit in an API-accessible account.

Order handling

- Limit orders are preferred over market orders for entries, with a defined maximum acceptable slippage — an order that cannot fill within tolerance is left unfilled rather than chased.
- Pre-trade sanity checks: order size vs. account equity, duplicate-order prevention (idempotency keys), and a market-hours check before any order is submitted.
- Post-trade reconciliation: every fill is compared against the intended price/quantity, and discrepancies are logged and alerted, not silently accepted.

10. Component 7 — Backtesting & Validation Framework

Purpose: Establish honest, defensible evidence of whether the system has a real edge before any capital is risked, and continue validating it afterward.

- **Walk-forward methodology:** rolling train / validate / test windows that move forward through time — never a single static train/test split, which leaks structure and overstates performance on time series.
- **Realistic cost model:** every backtest includes commission, bid-ask spread, and a slippage estimate as a function of order size relative to average daily volume — a strategy that only looks profitable at theoretical fill prices is not a strategy.
- **Benchmark comparison:** net-of-cost return and Sharpe ratio are compared against a buy-and-hold index benchmark *and* against a Fama-French factor regression, to confirm any apparent edge is not simply disguised market or factor exposure.
- **Regime testing:** the strategy is evaluated separately across distinct historical regimes (e.g. 2018 sell-off, 2020 crash and recovery, 2022 rate-hiking cycle, recent low-vol periods) rather than judged on one aggregate number — a strategy that only worked in one regime is not yet robust.

11. Component 8 — Monitoring, Logging & Retraining

Purpose: Detect model decay and system faults automatically, and provide a complete audit trail.

- **Immutable logging:** every signal, model output, risk-engine decision, order, and fill is logged with timestamps — needed for post-mortem analysis, performance attribution, and UK tax record-keeping.
- **Live-vs-backtest divergence tracker:** rolling comparison of realised live performance against what the validated backtest would have predicted for the same period; an alert fires if divergence exceeds a defined threshold, since this is the earliest reliable signal of overfitting or regime change.
- **Scheduled and triggered retraining:** Model A retrains on a fixed monthly cadence by default, with an additional trigger if rolling accuracy or Sharpe falls below a defined floor.
- **Kill switch:** a single control — reachable both automatically (on breach of the drawdown circuit breaker or a critical fault) and manually — that flattens all open positions and halts new order submission. This is tested in isolation before go-live, the same way a fire alarm is tested, not assumed to work.

12. Technology Stack

Layer	Suggested Technology	Notes
Language / core	Python 3.11+	Single language across research, backtest and live for code-path consistency
Data storage	PostgreSQL + TimescaleDB, or partitioned Parquet	Parquet is sufficient for a single-developer build; move to TimescaleDB if querying becomes a bottleneck
Data handling	pandas, NumPy, polars (optional, for speed)	polars worth adopting if dataset size grows beyond pandas' comfort zone
Modelling	scikit-learn, XGBoost / LightGBM, arch (GARCH), statsmodels	'arch' package is the standard Python GARCH implementation
Experiment tracking	MLflow	Already in your existing toolkit — reuse for model versioning and run comparison
Backtesting engine	vectorbt or backtrader, or a custom walk-forward harness	A custom harness gives the most control over point-in-time correctness, at the cost of more build time
Sentiment (optional)	FinBERT (via Hugging Face transformers)	Run offline/batch — no need for real-time inference at a daily/swing horizon
Execution / broker API	Interactive Brokers — ib_insync or native TWS API	ib_insync is the most ergonomic community wrapper around the native API
Scheduling / orchestration	cron for a single-developer build; Airflow/Prefect once complexity grows	Don't over-engineer orchestration before it's needed
Monitoring / alerting	Custom logging + a lightweight notifier (e.g. Telegram bot, as in your earlier signal-system design)	Reuse the Telegram alerting pattern for kill-switch and drawdown notifications
Dashboard	Streamlit	Reuse your EquityLens dashboard pattern for live monitoring, P&L and model-decay visualisation

13. Phased Build Roadmap

Each phase has an explicit exit criterion. A phase is not considered complete on a timeline basis alone — it is complete when its exit criterion is met, even if that takes longer than planned.

Phase	Indicative Duration	Activities	Exit Criterion
0 — Foundations	1–2 weeks	Set up data storage, ingestion jobs, point-in-time validation tests, repository structure	Historical and live data both pass validation checks consistently

Phase	Indicative Duration	Activities	Exit Criterion
1 — Features & research	2–3 weeks	Build the shared feature library; exploratory analysis of feature predictive power	Feature set is stable, documented, and produces identical output when run twice on the same inputs
2 — Models & backtest	3–4 weeks	Build Models A/B/C, the walk-forward backtesting harness, and the cost/slippage model	Strategy shows a positive, cost-adjusted edge across multiple distinct historical regimes — not just one favourable period
3 — Risk engine & execution	2–3 weeks	Build position sizing, stop-loss enforcement, drawdown breaker, IBKR integration, kill switch	Kill switch and drawdown breaker both verified to work correctly in isolated tests
4 — Paper trading	4–12 weeks	Run the full system on a paper/simulated account; compare live-paper performance to backtest expectations	Paper performance tracks backtest expectations within a defined tolerance, with no critical faults over the period
5 — Small live tranche	Ongoing	Deploy a small, strictly ring-fenced amount of real capital; monitor divergence tracker closely	Sustained behaviour consistent with paper trading over a meaningful sample of trades
6 — Gradual scale-up	Ongoing	Incrementally increase allocated capital, never as a reaction to a single good or bad week	N/A — continuous, with the option to scale back down at any time

14. Risk Control Summary

Control	Typical Rule	Purpose
Per-trade risk	0.5–1% of capital, volatility-scaled via ATR	No single trade can materially damage the account
Stop-loss	~2× ATR from entry, placed as a standing broker order	Bounds downside even if the system goes offline
Max leverage	Fixed cap, set before go-live and not adjusted reactively	Prevents leverage creep during winning streaks
Max concurrent positions	Fixed cap (e.g. 8–12)	Limits aggregate exposure and operational complexity
Sector concentration	Max % of capital per sector	Avoids concentrated, correlated bets disguised as diversification
Drawdown circuit breaker	Auto-halt new entries at a defined rolling drawdown (e.g. –8% / 20 days)	Forces a pause and manual review before losses compound

Control	Typical Rule	Purpose
Event blackout	Exclude/reduce size around earnings & major macro dates	Avoids gap risk the model was not trained to anticipate
Kill switch	Manual + automatic, flattens all positions	Final backstop against any uncontrolled failure mode

15. Pre-Live Deployment Checklist

Every item below should be checked off before Phase 5 (small live tranche) begins.

- Walk-forward backtest shows a positive, cost-adjusted edge across multiple distinct historical regimes.
- Backtest results compared against both a buy-and-hold benchmark and a factor (Fama-French) benchmark.
- Feature engineering code is shared, identical, and version-controlled between backtest and live paths.
- Paper trading run for a meaningful period, with live-paper performance tracking backtest expectations within tolerance.
- Stop-loss orders confirmed to be placed as standing orders at the broker, not only tracked internally.
- Drawdown circuit breaker tested in isolation and confirmed to halt new entries correctly.
- Kill switch tested in isolation — both manual trigger and automatic trigger — and confirmed to flatten positions.
- Maximum leverage, position count, and sector concentration caps are set and hard-coded, not configurable in the heat of the moment.
- Logging captures every signal, decision, order and fill with timestamps, sufficient for post-mortem and tax record-keeping.
- Capital allocated to the system is fully ring-fenced from other holdings and sized as capital you can afford to lose.

16. Appendix — Suggested Repository Structure

```
trading-system/
data/
  ingestion/      # scheduled jobs: price, news, macro feeds
  storage/        # DB models or Parquet schema definitions
  validation/     # point-in-time and schema checks
features/
  library.py      # shared feature code — imported by backtest AND live
models/
  direction_model/ # XGBoost / LightGBM training + inference
  volatility_model/ # GARCH fitting + forecasting
  factor_filter/   # rolling Fama-French regression
signal/
  generator.py     # ensemble logic, confidence threshold, universe filter
risk/
  sizing.py        # volatility-scaled position sizing
  controls.py      # stop-loss, leverage cap, drawdown breaker, kill switch
execution/
  ibkr_client.py  # broker API wrapper, order management
backtest/
  walk_forward.py  # rolling train/validate/test harness
  cost_model.py    # commission, spread, slippage assumptions
```

```
monitoring/  
  logger.py          # immutable audit logging  
  divergence_tracker.py # live-vs-backtest comparison  
dashboard/  
  streamlit_app.py    # live monitoring, P&L, model health  
tests/  
  test_point_in_time.py # regression tests for look-ahead bias  
  test_risk_controls.py # isolated tests for stop-loss/breaker/kill switch
```

This structure mirrors the component boundaries in Sections 4–11 directly, so that each architectural component maps onto exactly one module — making it straightforward to test, replace, or extend any single piece without disturbing the rest of the system.